DATA 201 - Week 11a

Regression with Ensemble Methods

Dr. Rebin Muhammad

Focus: Decision Trees, Random Forest (Bagging), Gradient Boosting

Learning Goals

By the end of today, you should be able to:

- Explain what an **ensemble model** is
- Explain how a **decision tree** makes predictions
- Describe the difference between **bagging** and **boosting**
- Read model metrics: **MSE** and **R^2**
- Compare Decision Tree, Random Forest, and Gradient Boosting in plain language

Warm-Up Question

If you wanted to estimate a house price, which would you trust more?

- One person's guess
- A class average of 30 guesses

Why?

(Keep this idea in mind. It connects directly to ensemble learning.)

Key Terms (Part 1)

- **Model**: a math tool that learns patterns from data
- **Feature**: an input variable (example: house size)
- **Target**: what we want to predict (example: house value)
- **Regression**: predicting a numeric value
- **Train/Test Split**: train on one part, test on unseen data

Key Terms (Part 2)

- **Decision Tree**: model that asks a sequence of split questions
- **Bagging**: train many models in parallel, then average
- **Random Forest**: many decision trees trained with bagging
- **Boosting**: train models in sequence; new models fix earlier errors
- **Gradient Boosting**: a common and powerful boosting method

Dataset Used in Class

We use the **California Housing** dataset.

Example features:

- Median income
- House age
- Average rooms
- Population
- Latitude / longitude

Target:

- Median house value (numeric)

Why Start with Decision Trees?

Because trees are:

- Visual
- Intuitive
- A strong foundation for understanding ensembles

Also important:

- Random Forest and Gradient Boosting are both built from trees

Decision Tree: Core Idea

A decision tree repeatedly asks yes/no questions such as:

- Is median income > 3.2?
- Is house age <= 25?
- Is average rooms > 5.5?

Each answer sends data down a branch.

At the end, a leaf node gives a prediction.

Tree Vocabulary

- **Root node**: first split (top of tree)
- **Internal node**: middle decision points
- **Leaf node**: final output value
- **Depth**: how many split levels from root to leaf
- **Split**: rule that partitions data into two groups

Simple Tree Walkthrough

Imagine one path:

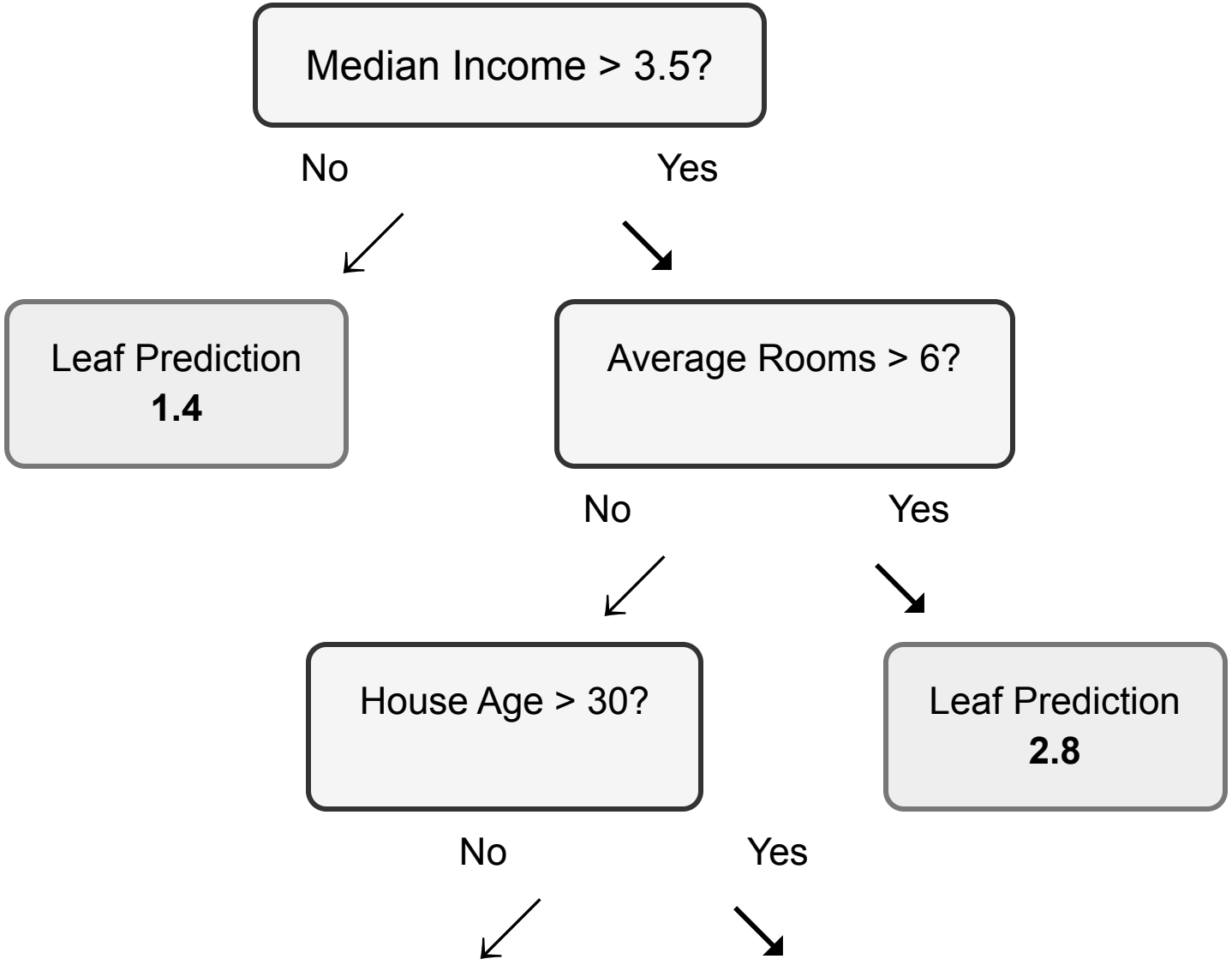
1. Is median income > 3.5? -> Yes
2. Is average rooms > 6? -> No
3. Is house age > 30? -> Yes

Leaf prediction: 2.1 (in dataset target units)

Takeaway:

- Trees turn complex prediction into a sequence of smaller choices.

Simple Tree Walkthrough



How Does a Tree Choose a Split?

For regression trees, the algorithm chooses splits that reduce error.

Informal idea:

- Before split: group has mixed target values
- After split: each child group is more similar

The model looks for splits that make children "cleaner" (less variance).

Decision Tree Strengths

- Easy to explain to non-technical audiences
- Captures nonlinear patterns
- Handles interactions automatically
- No feature scaling required

Decision Tree Weaknesses

- Can **overfit** if too deep
- Sensitive to small data changes
- One tree can be unstable

This is exactly why ensembles are useful.

Overfitting

Overfitting means:

- Model memorizes training details/noise
- Looks great on training data
- Performs worse on new data

For trees, warning signs:

- Very deep tree
- Very high training score, much lower test score

Controlling Tree Complexity

Common settings (hyperparameters):

- `max_depth`
- `min_samples_split`
- `min_samples_leaf`

Goal:

- Balance learning enough patterns without memorizing noise.

Quick Check (Think-Pair-Share)

Question:

Why might a very deep tree perform worse on test data than a moderate-depth tree?

(Discuss with partner for 1 minute.)

From One Tree to Many Trees

One tree = understandable, but unstable.

Many trees = usually more stable and more accurate.

This leads us to **ensemble learning**.

Big Idea: What Is an Ensemble?

An **ensemble** combines multiple models to make one final prediction.

Analogy:

- One student answer: maybe right
- Many student answers averaged: usually more reliable

Ensemble Family #1: Bagging

Bagging = Bootstrap Aggregating

Pipeline:

1. Sample training rows with replacement
2. Train one tree per sample
3. Average predictions

Main effect:

- Reduces variance (less jumpy predictions)

Bootstrap Sampling (In Plain Language)

"With replacement" means:

- A row can appear multiple times in one sample
- Some rows may be left out of that sample

Why useful?

- Creates many slightly different training sets
- Encourages diversity among trees

Random Forest = Bagging + Extra Randomness

Random Forest adds another trick:

- At each split, it only considers a random subset of features

Why this helps:

- Trees become less similar to each other
- Averaging works better when models are diverse

Random Forest Pros/Cons

Pros:

- Strong baseline model
- Robust performance
- Less overfitting than a single tree

Cons:

- Harder to interpret than one tree
- Can be slower than one tree

Ensemble Family #2: Boosting

Boosting builds models **sequentially**.

Idea:

- Model 1 makes predictions
- Model 2 focuses on Model 1's mistakes
- Model 3 focuses on remaining mistakes
- Continue and add them together

Gradient Boosting Intuition

Each new small tree learns from **residuals**:

- Residual = actual - predicted

If previous model underpredicted, new model pushes prediction up. If previous model overpredicted, new model pushes prediction down.

Gradient Boosting Hyperparameters

Important knobs:

- `n_estimators` (how many trees)
- `learning_rate` (how much each tree contributes)
- `max_depth` (complexity per tree)

Classroom intuition:

- Small learning rate + more trees = careful learning

Boosting Pros/Cons

Pros:

- Often excellent predictive accuracy
- Learns complex patterns gradually

Cons:

- More sensitive to tuning choices
- Can overfit if pushed too far
- Usually less interpretable

Bagging vs Boosting (Quick Compare)

Idea	Bagging (Random Forest)	Boosting (Gradient Boosting)
Training style	Parallel	Sequential
Main goal	Reduce variance	Reduce bias + errors
Typical stability	Very robust	Powerful but tuning-sensitive
Speed	Often faster	Can be slower

Metrics: How We Judge Models

MSE (Mean Squared Error)

- Average squared prediction error
- Lower is better

R^2 (R-squared)

- How much variation in target is explained
- Higher is better (closer to 1.0)

Metric Interpretation Example

Suppose:

- Model A: MSE = 0.52, R^2 = 0.62
- Model B: MSE = 0.41, R^2 = 0.70

Which is better?

- Usually Model B (lower error + higher explained variance)

Actual vs Predicted Plot

How to read it:

- Perfect predictions lie on the diagonal line
- Closer points = better predictions
- Wide spread = larger errors

Question:

- Which model's points are closest to the line?

Feature Importance (Careful Interpretation)

Feature importance tells us which features helped prediction most.

Important caution:

- Importance does **not** prove cause-and-effect
- It only reflects predictive usefulness in this model

Model Comparison Workflow

1. Train all candidate models on same training split
2. Evaluate all on same test split
3. Compare MSE and R^2
4. Inspect plots (actual vs predicted)
5. Choose model + explain tradeoffs

Classroom Discussion

If two models are close in performance, which would you choose?

Consider:

- Accuracy
- Interpretability
- Training time
- Ease of explanation to non-technical audience

In-Class Activity (12-15 min)

In groups of 2-3:

1. Compare Decision Tree, Random Forest, Gradient Boosting
2. Fill this sentence:
    - "Our best model is ___ because ___"
3. Name one risk/limitation of your chosen model
4. Share one practical use case

Guided Reflection Questions

- Why is one tree easier to explain than an ensemble?
- Why might Random Forest beat a single tree?
- Why might Gradient Boosting beat Random Forest on some datasets?
- Why should we always check test performance?

Common Student Mistakes

- Using only training score
- Forgetting MSE direction (lower is better)
- Assuming high R^2 means "perfect"
- Treating feature importance as causality
- Ignoring model tradeoffs (interpretability vs accuracy)

Where SVM Fits (Brief Extension)

SVM is a different model family.

In your notebook, SVM is also explored, including ensemble versions:

- Bagged SVR
- AdaBoost with SVR base estimator

Main message:

- Ensemble concepts can extend beyond trees.

Mini Glossary

- **Residual**: actual - predicted
- **Overfitting**: memorizes noise
- **Generalization**: performs well on new data
- **Hyperparameter**: setting chosen before training
- **Baseline model**: simple model used for comparison

Exit Ticket (Last 5 min)

Write brief answers:

1. In one sentence, what is ensemble learning?
2. One difference between bagging and boosting?
3. Why can deep trees overfit?
4. Which metric should go down? Which should go up?

Summary

Today we learned:

- Decision tree mechanics and vocabulary
- Why one tree can overfit
- How Random Forest stabilizes predictions
- How Gradient Boosting corrects errors step-by-step
- How to evaluate and explain model quality using MSE and R^2

Optional Homework

Using your Week 11 notebook:

1. Change one hyperparameter for each model
2. Record new MSE and R^2
3. Write 4-5 sentences comparing results
4. Explain one tradeoff between accuracy and interpretability